

GARCH Model and Hurst Exponent

1 Executive Summary

In this short article, we will use Garch model and fractional Brownian Model model to fit times series. In particular we will apply this to study three equity indices: FTSE, NIKKEI and S&P. We will see which model will provide better fitting to these equity index.

2 Non-Stationary Time Series

Garch model and Fractional Brownian motion are two different time series models which sometimes can be used to non stationary time series.

A weakly stationary times series is x_t where joint distribution of

$$x_{t+k_1}, x_{t+k_2}, \dots, x_{t+k_n} \quad (1)$$

is independent of t .

The well known stationary time series are family of ARMA models. However, many financial time series are non stationary, for example Garch and Brownian motion including fractionary Brownian Motions are well known ones. In this section we first review the Garch model family and Brownian motion models.

2.1 Arch Model

Let us review Arch model first. Garch model belongs to the family of Arch models. The Autoregressive Conditional Heteroskedasticity (ARCH) model was first introduced by Robert Engle in 1982. Arch model is a statistical model used primarily in time series analysis to model and forecast financial volatility.

For any time series y_t , we let $y_t = \mu + \epsilon_t$ where ϵ_t are white noise and $\epsilon_t \sim N(0, \sigma_t^2)$. Here σ_t is not a constant, unlike ARIMA models, σ_t may depends on time t . We can model σ_t by

$$\sigma_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + \dots + \alpha_p \epsilon_{t-p}^2$$

where $\alpha_i \geq 0$ are coefficients. Essentially this model can give us time dependent standard variations for the time series.

2.2 Garch Model

Bollerslev further proposed the Generalized Autoregressive Conditional Heteroskedasticity (GARCH) model in 1986, which extended the capabilities of ARCH by introducing more parameters to capture the dynamics of volatility. The Garch model has the following format

For a time series y_t , we model it as

$$y_t = \mu + \epsilon_t, \quad \epsilon_t \sim N(0, \sigma_t^2) \quad (2)$$

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^q \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^p \beta_j \sigma_{t-j}^2 \quad (3)$$

as we can see that the difference between Arch model and Garch model is that in Garch model we add the σ_{t-i} terms into the prediction of the σ_t terms as well. As we can see that under Garch model the times series y_t has a variance σ_t^2 depending on the history of σ_s^2 and on the history of y_s as well.

2.3 Fractional Brownian Motion

Fractional Brownian motion extends standard Brownian motion. The standard Brownian motion is a Gaussian Process, Markov process and at the same time a Martingale. It also has independent increments. Let W_t be the standard Brownian Motion, then we have

$$\text{Cov}(W_t, W_s) = \min(t, s) = \frac{1}{2}(t + s - |t - s|)$$

For a Fractional Brownian motion of exponent r we can define a new process W_t which satisfy

$$\text{Cov}(W_t, W_s) = \frac{1}{2}(|t|^{2r} + |s|^{2r} - |t - s|^{2r})$$

It can be verified that this process is an Gaussian process, but not a Markov process, neither a Martingale process. We can use the following code to generate the fractional Brownian motion. Here the parameter r is called the Hurst exponent.

When the Hurst exponent $r = 0.5$, it is the Standard Brownian motion where its increments are independent of each other. When the hurst exponent start to increase the time series will demonstrate trends where the paths tends to be more smooth. When the Hurst exponent starts to decrease the time series demonstrates reversals where the path tends to be more rough.

3 Applications to Equity Index

3.1 Fitting Garch and Hurst Parameters

We can use maximum likely hood method to fit the parameters of the Garch model. The Python has statsmodels library and we can use it to calculate all

parameters. The following code in Appendix can be used to fit the Arch/Garch model.

3.2 Application to Equity Index

We download the FTSE, NIKKEI, and S&P index for the past 20 years from 2004 to 2024. We then plot them in Appendix. Clearly they all demonstrate strong non stationarity.

3.3 Results

Finally we fit the Garch model for three indexes and the results are in Appendix. We also fit three index to Hurst index.

```
hurst_FTSE = 0.4217945391293815  
hurst_NIKKEI = 0.48835114928417644  
hurst_SP = 0.45983385355820083
```

4 Conclusion

We reviewed Non-Stationary time series models, Garch model and Brownian Motions. We fit the time series to Garch model. We have found that all three indices all demonstrate volatility cluster properties and Garch model can successfully explain them.

When we calculate their Hurst exponent, we see that FTSE have the roughest path, then NIKKEI and the last one is S&P.

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import datetime
from scipy import interpolate
from scipy.optimize import minimize
from scipy import optimize
from scipy.stats import norm
from sklearn.linear_model import LinearRegression

from arch import arch_model
import statsmodels.api as sm
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.graphics.tsaplots import plot_pacf
from statsmodels.tsa.stattools import acf
from statsmodels.tsa.stattools import pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.arima_process import ArmaProcess

import yfinance as yf
import pandas as pd

def generate_arima(length, ar_params, ma_params, d=0, scale=1):
    p = len(ar_params)
    q = len(ma_params)

    # Generate white noise
    white_noise = np.random.normal(0, scale, length + d)

    # Apply differencing if d > 0
    if d > 0:
        diff_series = pd.Series(white_noise).diff(d).dropna().to_numpy()
    else:
        diff_series = white_noise

    # Simulate ARMA process
    arma_series = np.zeros(length)
    for t in range(max(p, q), length):
        ar_part = np.dot(ar_params, arma_series[t-p:t][: -1]) if p > 0 else 0
        ma_part = np.dot(ma_params, diff_series[t-q+d:t+d][: -1]) if q > 0 else 0
        arma_series[t] = ar_part + ma_part + diff_series[t+d]
    return pd.Series(arma_series, name=f'ARIMA({p},{d},{q})')

# Generate ARIMA Model
def generateARIMA(p, q, n, ar_params, ma_params):
    u = np.random.normal(0, 1, n)
    x = np.zeros(n)
    epsilon = np.random.normal(0, 1, n)

    for i in range(max(p, q)):
        x[i] = epsilon[i]
```

```

        if (q > 0):
            for i in range(max(p, q), n):
                x[i] = np.dot(ar_params, x[i-p:i]) + np.dot(ma_params,
                    u[i-q:i]) + u[i]
            else:
                for i in range(p, n):
                    x[i] = np.dot(ar_params, x[i-p:i]) + u[i]
        return x

p = 4
q = 4
ar_params = np.ones(p) * 0.1 # AR coefficients (phi)
ma_params = np.ones(q) * 0.2 # MA coefficients (theta)
n = 10000
x = generateARIMA(p, q, n, ar_params, ma_params)
plt.plot(x)
plt.show()

nlags = 20 # Number of lags to compute
acf_values, confint, qstat, pvalues = acf(x, nlags=nlags, alpha
    =0.05, qstat=True)

plot_acf(x, lags=nlags, alpha=0.05)
plt.title("ACF Plot of the Time Series")
plt.xlabel("Lag")
plt.ylabel("Autocorrelation")
plt.show()

plot_pacf(x, lags=nlags, alpha=0.05)
plt.title("PACF Plot of the Time Series")
plt.xlabel("Lag")
plt.ylabel("Autocorrelation")
plt.show()

model = ARIMA(x, order=(p, 0, q))
model_fit = model.fit()
print(model_fit.summary())

ar_params = np.ones(p) * 0.1 # AR coefficients (phi)
ma_params = np.ones(q) * 0.2 # MA coefficients (theta)
y = generate_arima(n, ar_params, ma_params)

plt.plot(y)
plt.show()

model = ARIMA(y, order=(p, 0, q))
model_fit = model.fit()
print(model_fit.summary())

# Generate ARCH Model
def generateARCH(p, n, ar_params, omega):
    x = np.zeros(n)
    sigma2 = np.zeros(n)
    epsilon = np.random.normal(0, omega, n)

```

```

        for i in range(p):
            x[i] = epsilon[i]

        for i in range(p, n):
            sigma2[i] = np.dot(ar_params, x[i-p:i] * x[i-p:i]) + omega
            x[i] = np.random.normal(0, sigma2[i])
        return x

p = 4
n = 1000
ar_params = np.ones(p) * 0.1
omega = 0.04
x_arch = generateARCH(p, n, ar_params, omega)
plt.plot(x_arch)
plt.show()
model_arch = arch_model(x, vol='ARCH', p = p, mean='Zero', dist='
    normal')
results = model_arch.fit()
print(results.summary())

# Generate GARCH Model
def generateGARCH(p, q, n, a, b, omega):
    x = np.zeros(n)
    epsilon = np.random.normal(0, omega, n)
    sigma2 = np.zeros(n)
    for i in range(max(p, q)):
        x[i] = np.random.normal(0, omega)

    for i in range(max(p, q), n):
        sigma2[i] = np.dot(a, sigma2[i-p:i] * sigma2[i-p:i]) + np.
            dot(b, epsilon[i-q:i]*epsilon[i-q:i]) + omega
        x[i] = np.random.normal(0, sigma2[i])
    return x

p = 10
q = 2
n = 1000
ar_params = np.ones(p) * 0.08
ma_params = np.ones(q) * 0.2
omega = 0.04
y_garch = generateGARCH(p, q, n, ar_params, ma_params, omega)
plt.plot(y_garch)
plt.show()
model_garch = arch_model(y_garch, vol='GARCH', p = p, q=q, mean='
    Zero', dist='normal')
results = model_garch.fit()
print(results.summary())

# Calcualte Hurst Exponent
def hurst(ts):
    # Create the range of lag values
    lags = range(2, 100)

```

```

# Calculate the array of the variances of the lagged
# differences
tau = [np.sqrt(np.std(np.subtract(ts[lag:], ts[: -lag]))) for
       lag in lags]

# Use a linear fit to estimate the Hurst Exponent
poly = np.polyfit(np.log(lags), np.log(tau), 1)

# Return the Hurst exponent from the polyfit output
return poly[0]*2.0

#generate Brownian motion
x = np.random.normal(0, 1, n)
randomwalk = np.cumsum(x)
plt.plot(randomwalk)

#Test Brownina Motion Hurst Index
randomwalk_hurst = hurst(randomwalk)
print("Brownian Motion Hurst Exponent = ", randomwalk_hurst)

# Generate Fractional Brownian Motion
from fbm import FBM
n = 1000 # Number of steps
H = 0.2 # Hurst parameter (0 < H < 1)

# Generate fractional Brownian motion
fBm = FBM(n=n, hurst=H, length=1, method='cholesky').fbm()
plt.plot(fBm)
fBm_hurst = hurst(fBm)

indices = {
    "FTSE 100": "^FTSE",
    "S&P 500": "^GSPC",
    "Nikkei 225": "^N225"
}

start_date = "2004-02-09"
end_date = "2024-02-09"

data = {}
for name, ticker in indices.items():
    df = yf.download(ticker, start=start_date, end=end_date,
                     interval="1d")["Close"]
    data[name] = df

```

```

df_FTSE = data[ 'FTSE 100' ].values.reshape(-1)
df_NIKKEI = data[ 'Nikkei 225' ].values.reshape(-1)
df_SP = data[ 'S&P 500' ].values.reshape(-1)

plt.plot(df_FTSE)
plt.savefig(r'C:\Users\jians\Documents\Machine Learning\Code
Examples\Garch Model\FTSE.pdf', dpi=300, format='pdf',
bbox_inches = 'tight')
plt.show()

plt.plot(df_NIKKEI)
plt.savefig(r'C:\Users\jians\Documents\Machine Learning\Code
Examples\Garch Model\NIKKEI.pdf', dpi=300, format='pdf',
bbox_inches = 'tight')
plt.show()

plt.plot(df_SP)
plt.savefig(r'C:\Users\jians\Documents\Machine Learning\Code
Examples\Garch Model\SP.pdf', dpi=300, format='pdf',
bbox_inches = 'tight')
plt.show()

return_FTSE = np.diff(df_FTSE) / df_FTSE[0:-1]
return_NIKKEI = np.diff(df_NIKKEI) / df_NIKKEI[0:-1]
return_SP = np.diff(df_SP) / df_SP[0:-1]

hurst_FTSE = hurst(np.log(df_FTSE))
hurst_NIKKEI = hurst(np.log(df_NIKKEI))
hurst_SP = hurst(np.log(df_SP))

plt.plot(return_FTSE)
plt.savefig(r'C:\Users\jians\Documents\Machine Learning\Code
Examples\Garch Model\return_FTSE.pdf', dpi=300, format='pdf',
bbox_inches = 'tight')
plt.show()

plt.plot(return_NIKKEI)
plt.savefig(r'C:\Users\jians\Documents\Machine Learning\Code
Examples\Garch Model\return_NIKKEI.pdf', dpi=300, format='pdf',
bbox_inches = 'tight')
plt.show()

plt.plot(return_SP)
plt.savefig(r'C:\Users\jians\Documents\Machine Learning\Code
Examples\Garch Model\return_SP.pdf', dpi=300, format='pdf',
bbox_inches = 'tight')
plt.show()

print("hurst_FTSE =", hurst_FTSE, "hurst_NIKKEI =", hurst_NIKKEI,
      "hurst_SP =", hurst_SP)

```

```

plot_pacf(return_FTSE * return_FTSE, lags=nlags)
plt.title("PACF Plot of the Time Series")
plt.xlabel("Lag")
plt.ylabel("Autocorrelation")
plt.savefig(r'C:\Users\jians\Documents\Machine Learning\Code
Examples\Garch Model\FTSE-pacf.pdf', dpi=300, format='pdf',
bbox_inches = 'tight')
plt.show()

FTSE_garch = arch_model(return_FTSE, vol='GARCH', p = 1, q = 1,
mean='Zero', dist='normal')
results = FTSE_garch.fit()
print(results.summary().as_latex())

plot_pacf(return_NIKKEI * return_NIKKEI, lags=nlags)
plt.title("PACF Plot of the Time Series")
plt.xlabel("Lag")
plt.ylabel("Autocorrelation")
plt.savefig(r'C:\Users\jians\Documents\Machine Learning\Code
Examples\Garch Model\NIKKEI-pacf.pdf', dpi=300, format='pdf',
bbox_inches = 'tight')
plt.show()

NIKKEI_garch = arch_model(return_NIKKEI, vol='GARCH', p = 1, q = 1,
mean='Zero', dist='normal')
results = NIKKEI_garch.fit()
print(results.summary().as_latex())

plot_pacf(return_SP * return_SP, lags=nlags)
plt.title("PACF Plot of the Time Series")
plt.xlabel("Lag")
plt.ylabel("Autocorrelation")
plt.savefig(r'C:\Users\jians\Documents\Machine Learning\Code
Examples\Garch Model\SP-pacf.pdf', dpi=300, format='pdf',
bbox_inches = 'tight')
plt.show()

SP_garch = arch_model(return_SP, vol='GARCH', p = 1, q = 1, mean='
Zero', dist='normal')
results = SP_garch.fit()
print(results.summary().as_latex())

```

Graphs and Tables

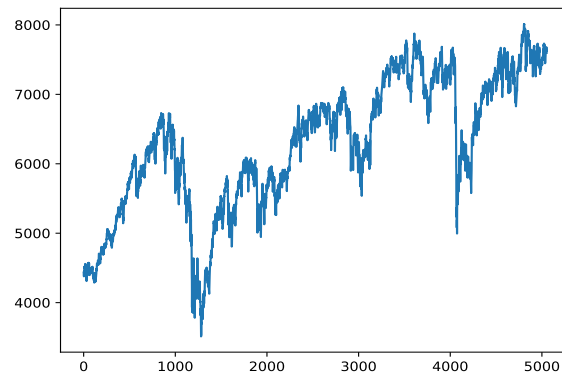


Figure 1: FTSE

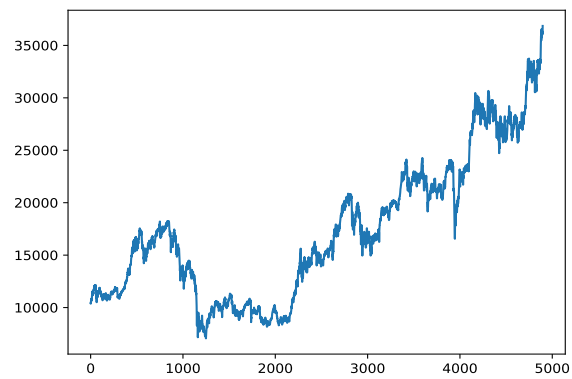


Figure 2: NIKKEI

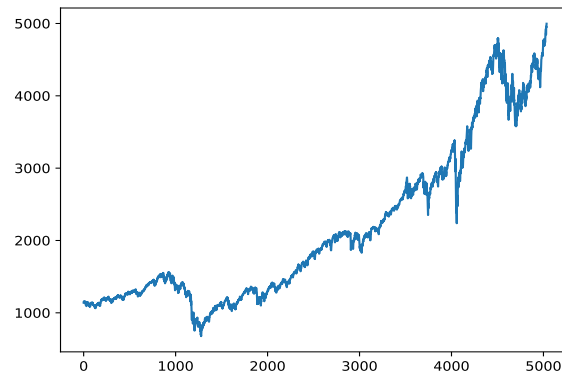


Figure 3: SP

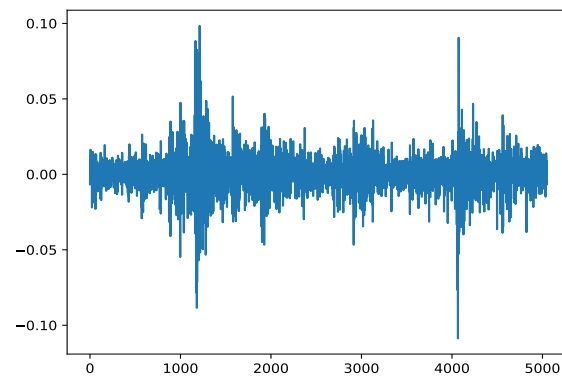


Figure 4: Return FTSE

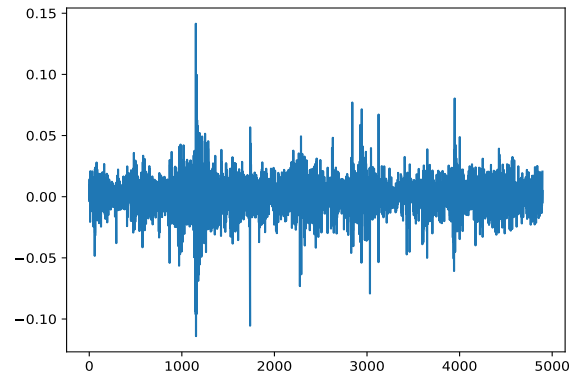


Figure 5: Return NIKKEI

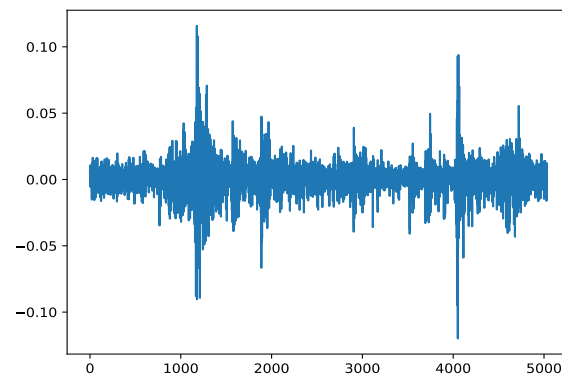


Figure 6: Return SP

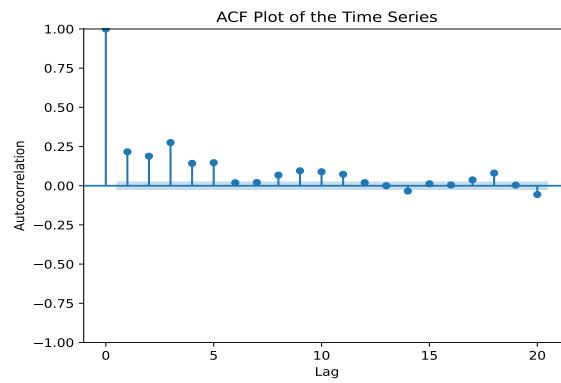


Figure 7: PACF FTSE

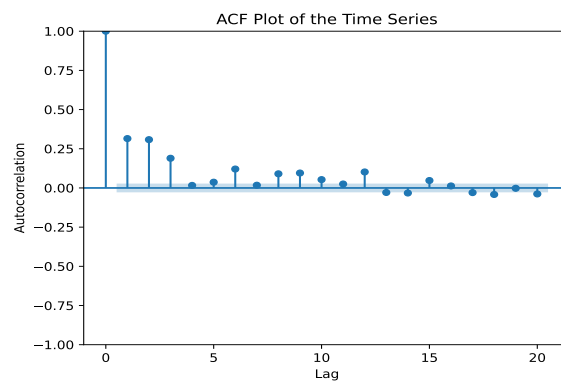


Figure 8: PACF NIKKEI

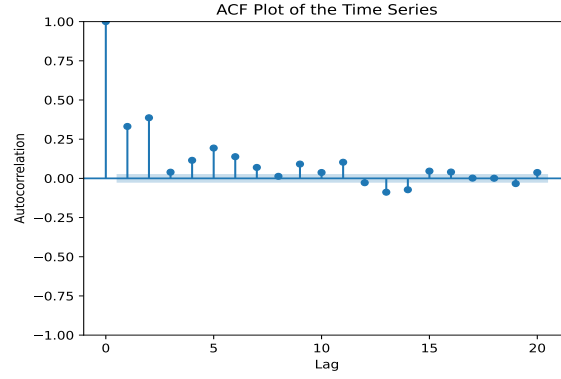


Figure 9: PACF SP

Table 1: FTSE - GARCH Model Results

Dep. Variable:	y	R-squared:	0.000
Mean Model:	Zero Mean	Adj. R-squared:	0.000
Vol Model:	GARCH	Log-Likelihood:	16590.4
Distribution:	Normal	AIC:	-33174.7
Method:	Maximum Likelihood	BIC:	-33155.1
		No. Observations:	5051
Date:	Sun, Feb 16 2025	Df Residuals:	5051
Time:	15:56:18	Df Model:	0

	coef	std err	t	P> t	95.0% Conf. Int.
omega	2.4621e-06	2.529e-12	9.735e+05	0.000	[2.462e-06,2.462e-06]
alpha[1]	0.1000	3.763e-03	26.577	1.239e-155	[9.263e-02, 0.107]
beta[1]	0.8800	1.203e-04	7315.481	0.000	[0.880, 0.880]

Table 2: NIKKEI - GARCH Model Results

Dep. Variable:	y	R-squared:	0.000		
Mean Model:	Zero Mean	Adj. R-squared:	0.000		
Vol Model:	GARCH	Log-Likelihood:	14576.6		
Distribution:	Normal	AIC:	-29147.2		
Method:	Maximum Likelihood	BIC:	-29127.7		
		No. Observations:	4896		
Date:	Sun, Feb 16 2025	Df Residuals:	4896		
Time:	15:57:07	Df Model:	0		
	coef	std err	t	P> t	95.0% Conf. Int.
omega	4.0771e-06	2.103e-13	1.939e+07	0.000	[4.077e-06,4.077e-06]
alpha[1]	0.1012	2.055e-03	49.246	0.000	[9.716e-02, 0.105]
beta[1]	0.8784	4.283e-03	205.088	0.000	[0.870, 0.887]

Table 3: SP - GARCH Model Results

Dep. Variable:	y	R-squared:	0.000		
Mean Model:	Zero Mean	Adj. R-squared:	0.000		
Vol Model:	GARCH	Log-Likelihood:	16479.6		
Distribution:	Normal	AIC:	-32953.1		
Method:	Maximum Likelihood	BIC:	-32933.6		
		No. Observations:	5034		
Date:	Sun, Feb 16 2025	Df Residuals:	5034		
Time:	15:57:32	Df Model:	0		
	coef	std err	t	P> t	95.0% Conf. Int.
omega	2.9036e-06	5.236e-12	5.546e+05	0.000	[2.904e-06,2.904e-06]
alpha[1]	0.1000	1.390e-04	719.287	0.000	[9.973e-02, 0.100]
beta[1]	0.8800	2.754e-03	319.560	0.000	[0.875, 0.885]